IP:

		Subnet mask	CIDR
Class A :	1.0.0.0 – 126.255.255.255	N H H H 255.0.0.0	/8
Class B :	128.0.0.0 – 191.255.255.255	N N H H 255.255.0.0	/16
Class C :	192.0.0.0 – 223.255.255.255	N N N H 255.255.255.0	/24

IPV4 : 4 octets × 8 bits = 32 bits

→ 1 octets = 8 bits ∴ 2^8 = 256 values

i.e. 0 to 255 [255.255.255.255]

→ **Host bits = 32 − CIDR**

Ex:- 192.168.1.0/24 → Network bits

∴ Host bits = 32 − 24 = 8

→ **Total IPs = 2^(Host bits)**

Ex:- 192.168.1.0/24

Host bit = 8

∴ Total IPs = 2^8 = 256

But 1st IP = network address [192.168.1.0]

last IP = Broadcast address [192.168.1.255]

∴ Available ~~Hosts~~ are = 256 − 2 = 254

→ AWS has 5 IP addresses reserved per subnet

Ex:- 10.0.1.0/24

Total = 256

But available = 256 − 5 = 251 IPs

→ Ex:- 10.0.0.0/24

Host bit = 32 − 24 = 8

Total IPs = 2^8 = 256

Usable IPs = 256 − 2 = 254

Aws usable = 256 − 5 = 251

→ **Number of Subnets = 2^(Borrowed bits)**

Ex:- Need 3 subnets ∴ 2^2 = 4 ∴ need 2 bits

Need 10 subnets ∴ 2^4 = 16 ∴ 4 bits

Private IP

A – 10.0.0.0 – 10.255.255.255

10.0.0.0/8

B – 172.16.0.0 – 172.31.255.255

172.16.0.0/12

C – 192.168.0.0 – 192.168.255.255

192.168.0.0/16

* 127.0.0.0/8 – reserved for loopbacks address

* CIDR tells us:

1. Network size
2. Number of available IPs
3. Subnet boundary.

32
26
6

2^6 = 64

64 − 2

62

VPC

→ Logically isolated virtual network that we create inside AWS. to launch cloud resources.

Cloud Watch

↳ monitoring and observability service used to collect, monitor, visualize, analyze and respond to metrics, logs and events from AWS application.

→ Monitoring - continuously observing the health, performance, availability and behaviour of aws resources and application.

→ Observability - ability to understand the internal state of a system by analysing its metrics, log, traces and data.

→ Metrics

↳ Metric - a numerical measurement collected over time that represents the performance of resource.

↳ Period - length of time over which cloudwatch aggregate metric data for evaluation.

→ Logs

↳ is a service capability that collects, stores, monitors, and analyze data from AWS resources, applications

↳ Log groups :- container that organize related log stream and allow common setting such as retention and access control to be applied.

↳ Log stream - sequence of log events from a particular source or application instance with in a log group.

↳ Log Events - individual record in a log stream, normally containing a timestamp and a log message.

↳ Log Retention - specifies how long cloudwatch logs keeps logdata before automatically deleting it.

↳ Log Insight - interactive log-analysis capability that allows user to query, search, filter and visualize log data.

→ Alarms :

↳ monitors a metric and changes its state when the metric meets a configured condition or threshold.

↳ Threshold - value against which a metric is compared to determine whether an alarm condition has been met.

↳ Evaluation period - amount of metric period that cloudwatch evaluate to determine an alarm's state.

↳ Alarm State :

1. OK -
2. ALARM -
3. INSUFFICIENT_DATA

→ Visualization :

↳ Dashboard - customizable visual interface that display metric, graph, alarm and other monitoring info in one place.

↳ Widget - individual visual component on a cloudwatch dashboard.

→ Events :

↳ Record that describes something that happened in system.

↳ Eventbridge : serverless event bus service that receives events from aws services and routes them to target.

→ RUM : [Real User Monitoring]

↳ collect info about the performance and behaviour of real users interacting with application.

→ Agent :

↳ software installed on supported server to collect additional system metric, logs and other then send to cloudwatch.

→ Alert :

↳ inform / notify a response generated when a monitored condition indicates that attention may be required.

→ Agent used

1. EC2, server, Linux server, On-premise server, VM

don't use

1. S3, RDS, lambda, dynamoDB, SNS, SQS, API Gateway, LB

Application Load Balancer

↳ operates at OSI layer 7.

↳ distributes application traffic.

↳ ALB understand content of HTTP/HTTPS requests and can make routing decisions based on that info.

→ Path-based Routing - routing traffic according to URL path.

→ Host-based Routing - route based on host name.

→ HTTP header-based Routing - routing decision using HTTP header.

→ ALB perform health checks on target

→ ALB supports TLS termination.

→ Cross-Zone Load Balancing :

↳ allow traffic to be distributed across target in different AZ.

Network Load Balancers

↳ handles network-level connection, Layer-4

↳ main protocols - TCP, UDP, TLS

↳ Transport-level connection routing

↳ supports TLS Listener

↳ provides static IP address for LB, and can also use Elastic IP address for internet-facing NLB

↳ Very high throughput, very low latency, large number of connections, TCP/UDP application.

↳ Health check on TCP, HTTP, HTTPS.

↳ NLB can route traffic to supported target types such as : EC2, IP address, ALB.

GLB

↳ designed to deploy, scale and manage virtual network appliances such as firewalls, IDS, IPS, deep packets inspection.

1. distribute traffic among the firewalls
2. automatically scale the appliances
3. Maintain network traffic flow
4. Make the appliances highly available.

↳ appliances could be:

1. Firewall
2. IPS, IDS
3. network/security appliances
4. Packet inspection

→ GLB Endpoints is type of VPC endpoint that allow traffic from your VPC to reach GLB.

↳ GLBE uses AWS Privatelink technology

→ GLB uses GENEVE protocol for encapsulation

→ GENEVE port - UDP 6081

[Generic Network Virtualization Encapsulation]

→ Flow Stickiness - GLB uses flow-based distribution so traffic belonging to a flow remain associated appropriately with same appliance. [Stateful appliances]

→ Stateful firewall remembers connection information.

→ Service insertion means inserting a network service into the traffic path.

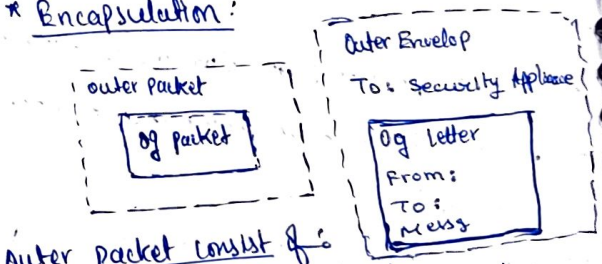
→ GLB provides transparent network application insertion.

→ In 5 tuple - Protocol tells "what kind of traffic is this?" which helps GLB to determine which appliance should handle a particular flow.

User
↓
Internet
↓
Internet Gateway
↓
Route Table
↓
GLB Endpoint
↓
GLB
↓
Select healthy appliance [Firewall]
↓
GENEVE / UDP 6081
↓
Firewall
↓
Decapsulation
↓
Inspection

- Deny → Drop
- Allow → continue → GLB → GLBE → Application.

* Encapsulation:



Outer packet consist of:
Outer IP header, UDP, Src/Dst ports, UDP dst = 6081, GENEVE header, Og Packet

Inside/original packet consist: 5 tuples

← Outer Envelop

EventBridge

↳ Serverless event bus service that connects application, AWS services, 3rd party apps by receiving events, filtering them using rules and routing them to appropriate targets.

→ Event – something that happn in system.

→ EDA – s/w architecture in which components communicate by producing and responding to events.

* Source → Event → Event bus → Rule → Target → target is destination that eventbrige send an event to which a rule matches
  - Source: EC2, Lambda, S3 etc.
  - Event bus: router that receives event and delivers them to zero or more targets based on rules.
  - Rule: ↳ represents/determine which event should be sent to which target.
  - Target: Ex:- Lambda, SQS, SNS, API, Kinesis, ECS, etc.

→ Eventbridge Pipe – provides point to point integration b/w single event source and a single target.

→ Eventbridge – will let you route events from one application to another based on rules.

VPC [Virtual Private Cloud]

↳ we define IP range using CIDR

↳ Subnet belongs to one AZ, but VPC can have subnets in multiple AZ.

→ Public subnet – route table connected to IGW.

→ Private subnet – doesn't have direct connection to IGW.

→ IGW – provide path b/w VPC and internet.

→ Route Table – tells the network where to send traffic.

→ NAT Gateway – allow resources in private subnet to initiate outbound connections to internet without giving those resources direct inbound internet access.

↳ NAT is placed in public subnet to route through the IGW to reach internet.

→ Security Group – acts as firewall for resources. (→ network - interface level)

↳ stateful, allow incoming and outgoing traffic.

→ NACL – stateless, support both allow and deny rules

↳ works at subnet level

→ VPC Endpoints – private connectivity b/w resources in VPC and supported AWS services.

→ Gateway Endpoints – used for S3, dynamoDB

→ Interface Endpoints – use AWS private links and create a network interface with private IP address in our subnet.
Ex:- KMS

→ VPC Peering – connection b/w two VPCs.

→ Transit Gateway – acts as central networking hub. multiple VPCs and other network connections can connect through it.

→ VPN – AWS site to site VPN provides an encrypted connection b/w an on-premises network and AWS.

→ Direct Connect – dedicated network connection to AWS.

→ VPC Flow Logs – Help us capture information about network traffic to and from network interface in VPC.

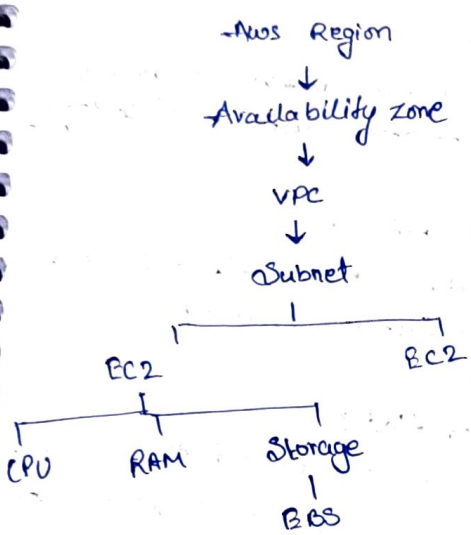
EC2

-> Service that provide resizable compute capacity.

-> Allows you to create and run virtual servers in the aws cloud.

-> Aws handles the underlying infras.

-> User is responsible for "guest OS and everything running inside instance".



1) AMI : [Amazon Machine Image]

↳ template used to create your EC2 instance.

↳ Ex:- Amazon Linux, Ubuntu, Red hat, Windows

↳ Contains : OS, root volume config, Software

2) Instance type :

↳ determine hardware resources available to your virtual machine.

↳ Ex: t3.micro, t3.small, m7i.large, etc..

↳ determines things like :

vCPUs, memory, network capabilities, Storage, architecture, processor, performance.

* Instance Families :

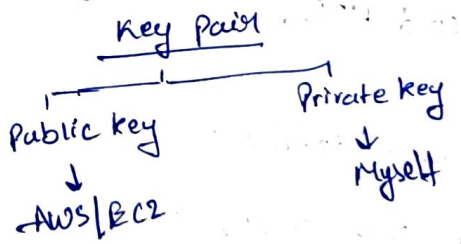
↳ different workloads need different resources.

↳ Types :

1. General Purpose (T, M)
   ↳ Web servers, development, small applications, general workload
2. Compute Optimized (C)
   ↳ CPU-intensive applications
   ↳ batch processing
   ↳ Compute heavy workloads
3. Memory Optimized (R, X)
   ↳ large database
   ↳ in-memory application
   ↳ caching
4. Storage Optimized (I, D, H)
   ↳ high I/O workload
   ↳ large local dataset
   ↳ storage-heavy application
5. Accelerated computing (P, G, Inf, Trn)
   ↳ machine learning, GPUs;
   ↳ AI workloads.

3) Key Pair :

↳ Securely connect EC2 instance



4) Network Settings :

1. VPC
2. Subnet
3. Auto assign Public IP
4. Security Groups.

5) Security Groups :

↳ virtual firewall

↳ Inbound rules – traffic coming towards EC2.

↳ Stateful [outbound connection is automatically allowed]

6) Configure Storage :

↳ EC2 normally uses EBS for persistent block storage.

* EBS volumes :

1. gp3
   ↳ general purpose SSD
   ↳ very commonly used
2. gp2
   ↳ older version
3. io2
   ↳ Designed for high performance, high-durability workloads requiring high IOPS.
4. st1
   ↳ Throughput optimized SSD
5. sc1
   ↳ cold HDD.

7) Advance details :

① IAM instance profile

↳ EC2 can have IAM role

↳ Instead of putting Aws access keys inside code, you attach IAM role

② Metadata options :

↳ EC2 has important service called Instance Metadata Service – IMDS

↳ Instance can retrieve info about itself.

↳ Modern EC2 uses IMDSv2, which uses a session-oriented token mechanism

③ Tenancy :

1. Shared Tenancy
   ↳ Instance runs on Aws hardware shared with other customers at underlying h/w.
2. Dedicate Instance :
   ↳ Your instance runs on h/w dedicated to a single customer, while Aws manages placement.
3. Dedicated Host :
   ↳ Physical server dedicated to your use and can manage instance placement.

④ Placement Groups :

↳ how instance are physically/logically placed

↳ Types :

1. Cluster :
   ↳ Instances are placed close together
   ↳ Useful for :
      1. High network performance
      2. Low latency
2. Spread :
   ↳ Instances are spread across distinct underlying h/w.
   ↳ Useful for :
      1. Reducing correlated h/w failure risk.

3. Partition:
↳ Instance separated into partitions
↳ Useful for large distributed system:
Hadoop, Kafka, Cassandra

4. Precision Time:
↳ about clock synchronization

(5) Purchasing Option:

1. On-Demand:
↳ pay for what you use without long-term commitment.
↳ Good when workload is unpredictable.

2. Reserved Instance:
↳ Commit to a config/term for a discount compared with equivalent on-demand usage.

3. Saving Plans:
↳ Commit to certain amount of compute spending for a term in exchange for discounted rates.

4. Spot Instance:
↳ Use spare AWS capacity at potentially large discounts, but AWS can interrupt the instance when capacity is needed.
↳ Good for -
↳ Batch jobs, Fault-tolerant workloads, stateless processing

* Hypervisor:
↳ EC2 relies on AWS virtualization technology.
↳ AWS hypervisor - AWS Nitro.

* EC2 Status Checks:
- 2/2 checks passed
1. System status check
↳ Checks underlying AWS infra.
↳ AWS hardware, AWS networking, AWS host infrastructure.

2. Instance status check
↳ check instance itself
↳ Indicate problem:
↳ OS boot failure
↳ Network config problems
↳ Instance-level issues.

- 3/3 checks passed
↳ It include "Attached EBS status check"

* Elastic IP:
↳ stable public IPV4 address

S3

[Simple Storage Service]

↳ Used to store and retrieve data as object.

AWS
↓
S3
↓
Bucket
↓
obj 1 — obj 2
↓
Data

* Bucket name should be globally unique because bucket name become part of the public S3 DNS namespace and URL.

↳ Important Concept:

1. Bucket : Container for object
2. Object : The actual file/data stored
3. Key : unique name/path used to identify an object in bucket.

* S3 is regional, but bucket names are globally unique.

* How to create S3 bucket :

S3
↓
Create Bucket
↓
Choose Region
↓
Unique Bucket name
↓
Object Ownership
↓
Block Public Access
↓
Versioning
↓
Tags
↓
Default Encryption → Advanced setting → Create Bucket → Bucket Created → Upload Objects

* Object Ownership

1. ACLs disabled
2. Bucket owner enforced

↳ ACL - Access Control List

Before S3 used ACL but now AWS recommend to use IAM Policies

↳ Bucket Owner enforced -

bucket owner owns the object in bucket and ACLs are disabled

* Block public Access

↳ 4 imprtnt rules/setting :

1. Block public ACLs - prevent new public ACL from being created
2. Ignore public ACLs - Ignore public ACLs when evaluating access.
3. Block public bucket policies -
4. Restrict public bucket policies

* Bucket Versioning

↳ allows S3 to maintain multiple versions of an object.

↳ Enable versioning, can retain previous versin.

* Tags [Key value pairs]

↳ useful for :

↳ Cost allocation, organization, Automation, Resource management

* Default Encryption

↳ S3 supports encryption of objects at rest.

↳ Types : → AWS manages encryption automatically

1. SSE-S3 → straightforward
2. SSE-KMS → more control over encryption keys
3. DSSE-KMS

Advanced security control encryption key. used by enterprise

↓ dual layer server-side encryption with AWS KMS

* Advanced settings

• Object Lock :

↳ Object Lock is used when you need objects to remain Immutable for a retention period.

* Delete Marker :

↳ If file.txt exist in bucket, I want to delete it, when versioning is enable the S3 doesn't simply destroy it, instead it creates "Delete Marker"

* Versioning can increase S3 storage costs.

* Bucket Key :

This is S3 feature that can reduce the number of requests to this KMS for SSE-KMS encryption, reduces KMS request cost.

* Object lock modes :

1. Governance mode :
   specially authorized users can potentially bypass retention controls.
2. Compliance mode :
   Retention is stricter, and even highly privileged user cannot simply bypass the retention period.

* Is S3 a folder structure ?

No, It uses a flat object namespace.

* S3 object consist of Key, Data, metadata, tags, version ID.
  ↳ Info about the objects

* S3 Storage Classes :

1. S3 Standard
   ↳ used for frequently accessed data
   ↳ High availability, high durability, Frequent access, general-purpose storage
2. S3 Intelligent-Tiering
   ↳ Useful when you don't know how frequently your objects will be accessed
   ↳ S3 automatically moves objects between access tier based on access patterns.
   ↳ reduce storage costs without manually predicting access pattern.
3. S3 Standard-IA [Infrequent Access]
   ↳ must remain quickly accessible
   ↳ isn't accessed frequently.
   ↳ Storage is cheaper than standard, but retrieval costs apply.
4. One Zone-IA :
   ↳ similar to Standard-IA but data stored in a single Availability zone
   ↳ cost less, but less resilience to an AZ-level failure than multi AZ storage
   ↳ use only when trade-off is acceptable
5. Glacier Instant Retrival :
   ↳ For archive data that is rarely accessed but need fast retrival when requested.
6. Glacier Flexible Retrieval :
   ↳ Designed for archival data where retrieval can take longer.
   ↳ you can choose retrival options depending on how quickly you need data.
7. Glacier deep Archive :
   ↳ Designed for very long-term archival
   ↳ optimized for very infrequent access.

* S3 is highly durable :

↳ Yes, extremely high durability by redundantly storing data across AWS infra.

* Durability v/s Availability :

D – How likely is my data to survive without being lost ?

A – How often can I access the data when I request it ?

* S3 replication :

1. Same-Region Replication (SRR)

Bucket A (Mumbai) → Bucket B (Mumbai)

2. Cross-Region Replication (CRR)

Bucket A (Mumbai) → Bucket B (Ohio)

↳ useful for :

1. Disaster recovery
2. Compliance
3. Geographic redundancy
4. Lower-latency access in another region

* S3 Lifecycle :

↳ As my object get older, automatically move them to cheaper storage or delt them when I no longer need them.

↳ Flow :

New log

↓

[Frequent] S3 Standard

↓ [After 30 days]

[less frequent] S3 Standard-IA

↓ [After 90 days]

[Rarely use] S3 Glacier Flexible Retrieval → [After 365 days] → Delete

↳ S3 2 important actions :

1. Transition – object one class to another
2. Expiration – Delete object

* Bucket Policy :

↳ is a resource-based policy attached to an S3 bucket.

↳ defines who can access this bucket, what they can do, and under what conditions.

* S3 Common Permissions :

1. GetObject – Download / read an object
2. PutObject – Upload an object
3. DeleteObject – Delete
4. ListBucket – list obj in bucket

* S3 Presigned URL :

↳ when bucket is private, but ~~you~~ want a user to download one particular file temporarily, then we creat Presigned URL

Storage classes:

Class		
S3 Standard	Use frequently	website images
S3-Standard IA	Use occasionally	Monthly reports
S3-Intelligent Tiering	Dont know how often you'll use it	User docs with unpredictable access.
S3 One Zone IA	Rarely used, okay with one AZ	Re-cretable backups.
S3 Glacier Instant Retrieval	Archive but need it quickly	Medical/legal records.
S3 Glacier Flexible Retrieval	Archive, retrieval can take longer	Old backups
S3 Glacier deep Archive	Very rarely accessed long-term archive	7-10 years compliance records

Standard-IA

1. Data accessed occasionally
2. Storage cost - Higher
3. Retrieval cost - Lower
4. Type - Monthly report, backups
5. Minimum duration - 30 days

S3 Glacier Instant Retrieval

1. Archive data that is rarely accessed but must be retrieved
2. Storage cost - lower
3. Retrieval cost - Higher
4. old medical record
5. 90 days.

S3 Lab

Bucket type – General purpose

↓

support - img, vid, pdf, website hosting, backup, Logs, IAM, versioning, lifecycle policies

Bucket namespace –

↳ Global – must be unique

↳ Account – AWS will recommend

Block public access

↳ protect files from the internet, nobody can access them publicly.

IAM

[Identity Access Management]

- Who can access which resource?
- What actions they can perform?
- Under what conditions access is allowed or denied?

→ It controls programmatic access, temporary credentials, roles and permissions used by application and AWS services.

→ **Identity** : An entity that can be recognized in an access request, such as an IAM user or an assumed IAM role.

→ **Authentication** : Proving who you are?

→ **Authorization** : What are you allowed to do?

→ **Policy** : set of rules that describes allowed or denied actions, resources and possibly conditions.

→ Authentication doesn't automatically grant administrative permissions.

* **IAM User** – Identity created inside an AWS account

↳ User contains :

1. User name
2. ARN [uniquely identifies the user in] → [Amazon Resource Name]
3. Console password
4. Access keys
5. Attached policies
6. Group memberships
7. Permissions boundary
8. Tags

↳ IAM user ~~ARN~~ names must be unique in their account.

* **IAM Groups** :

↳ collection of IAM users.

↳ You attach permission to the group, and its users inherit those permissions.

↳ **Rules of groups** –

1. group is not a login identity
2. can contain IAM users, not other groups.
3. can't sign to AWS console.
4. can't assume a role as if it were a user.
5. User can belong to multiple groups.

* **IAM Roles** :

↳ is an identity that defines a set of permissions and can be assumed by a trusted principal.

IAM Role for EC2 to read S3 file:

1. Create an IAM role for EC2 workload.
2. Attach appropriate S3 read policy to the role
3. Configure the role's trust policy to allow the EC2 service to assume it.
4. Attach role to EC2 instance through an instance profile.
5. Application obtains temporary credentials through EC2 instance metadata service.
6. Application uses those credentials to request the S3 object.

* Two essential parts of Roles:

1. **Who is allowed to assume the Role?**
   ↳ This is controlled by the role's trust policy
2. **What can the role do after it is assumed?**
   ↳ This is controlled by the role's permissions policies and any applicable restrictions.

↳ These 2 things are not **interchangeable**.

* Common Roles:

1. Cross-account role – Allow trusted principal of another account
2. Federated access role – Allow an externally authenticated user to obtain AWS access
3. Service-linked role – Let an AWS service perform specific operation on your behalf
4. Deployment role – Let an authorized CI/CD system deploy resources.
5. Read-only role – Allow users or workloads to inspect resources without modifying them.

* IAM Policy:

↳ document containing permission rules

↳ Main policy elements:

1. Version – policy language version
2. Statement – contains one or more permission
3. Effect – Allow or Deny
4. Action – AWS API operation
5. Resource – specifies the resource to which statement applies.
6. Principal – Identifies who is granted or denied access in a resource-based policy or who is trusted in role trust policy.
7. Conditions – Extra requirements [MFA, Region, tags ...]

↳ Types of IAM Policies:

1. Identity based – attached to an IAM user, group, role, to define what that identity can do.
2. Resource based – attached to a supported resource to define which principals may access it and what they may do.
3. Permissions boundaries – limit the maximum permissions that identity based policies can grant to a user or role.
4. Session Policies – for temporary sessions
5. Service control policies – ↳ for principals
6. Resource control policies – ↳ for resources
7. VPC endpoint policies – control which principal and resource can access VPC.
8. Access control list – service-specific resource access mechanism.

* Inline Policies:

↳ embedded directly into one IAM role, user, group.

↳ use when permission is intentionally tied to one identity and should have the same lifecycle as that identity.

* MFA:

↳ adds another factor to authentication.

* Least Privilege:

↳ Granting only the permission required to perform a specific task.

* Trust Policy:

↳ control which principals are trusted to assume the role.

* Permission Policy:

↳ grand permission to retrieve the specified object, subject to other applicable authorization requirements.

Lambda :-

↳ Serverless
↳ need not to manage the underlying servers

↳ Your are responsible for:
1. Writing code
2. Configuring IAM permissions.
3. Setting memory and timeout values
4. Managing application dependencies
5. Monitoring errors and performance
6. Control / Manage costs.

* **AWS Lambda function :**
↳ unit of code that Lambda executes

* **Runtime :**
↳ provide the environment needed to execute your code in particular programming language.

* **Handler :**
↳ It is the entry point Lambda calls when your function is invoked.

↳ handler format : "filename.function_name"

* **Event :**
↳ an event is the input supplied to your function when something invokes it.

↳ It might contains :
- user's API request.
- Info about an object.
- A message or notif or event.

* **Context object :**
↳ context parameter contains info about the current Lambda invocation and execution environment

* **Response :**
↳ It is the value your handler returns to the caller on invoking integration.

* **IAM execution role :**
Two different permissions —
1. Who can invoke Lambda?
Invocation permission determines who or what can call the function.
2. What can lambda access?
Execution role determines what the function can do when it calls AWS services.

* **Triggers :**
↳ an event source or integration that cause a Lambda function to be invoked.

* **Timeout :**
↳ max duration lambda allows an invocation to run before stopping it.

* **Memory :**
↳ amount of memory allocated to function execution environment.

* **cold start :**
↳ additional initialization work when lambda needs to prepare an execution environment for an invocation.

* **Invocation types :**
1. Synchronous - caller waits for lambda's invocation
↳ caller waits for the function to complete and receive a response.
Ex:- API Gateway calling Lambda
2. Asynchronous - caller sends the request and continues without waiting, Lambda process it later
Ex:- S3 event triggering Lambda.

3. Event source mapping:

↳ It is a configuration that connects an event source (SQS, Kinesis, Dynamo) to a Lambda function.

↳ Some AWS services do not directly invoke Lambda. Instead, Lambda needs to continuously check (poll) these services for new message or records.

↳ ESM handles this polling automatically.

* $LATEST:

↳ when you update a Lambda function code or configuration without publishing a version, you are working with its mutable, unpublished config, referred as $LATEST.

* Published Version:

↳ An immutable snapshot of a function's code and qualifying config.

* Alias:

↳ named pointer to a published version.